ASSIGNMENT-11.5

2303A51600

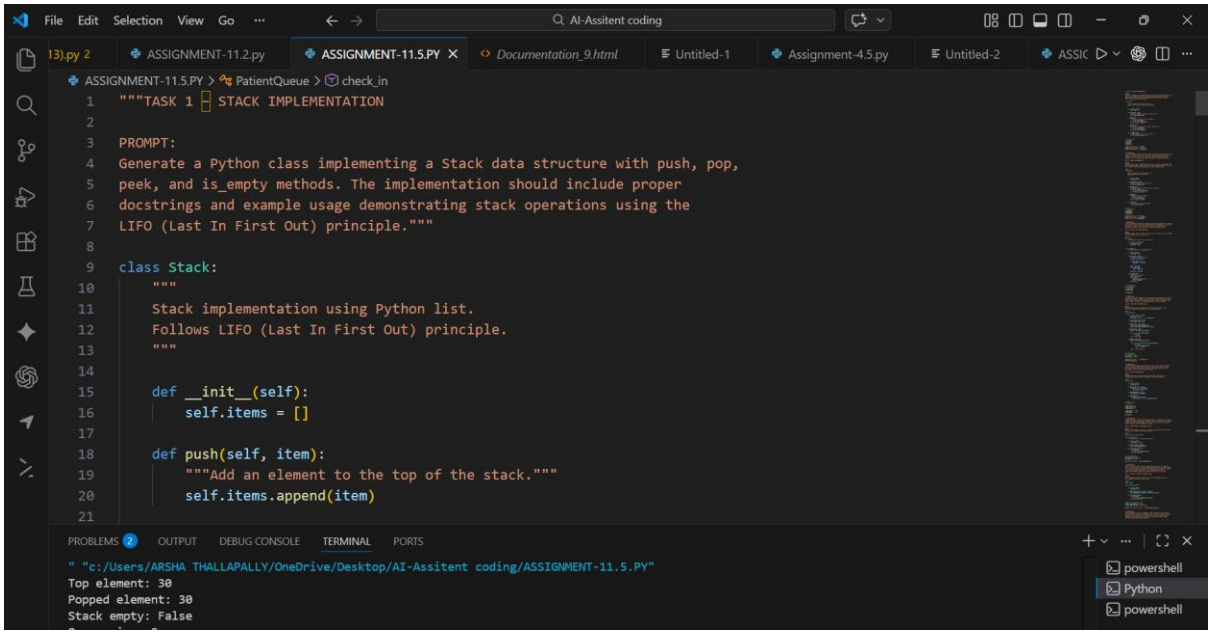
BATCH-29

TASK 1 – STACK IMPLEMENTATION

PROMPT:

Generate a Python class implementing a Stack data structure with push, pop, peek, and is_empty methods. The implementation should include proper docstrings and example usage demonstrating stack operations using the LIFO (Last In First Out) principle.

CODE:



OBSERVATION:

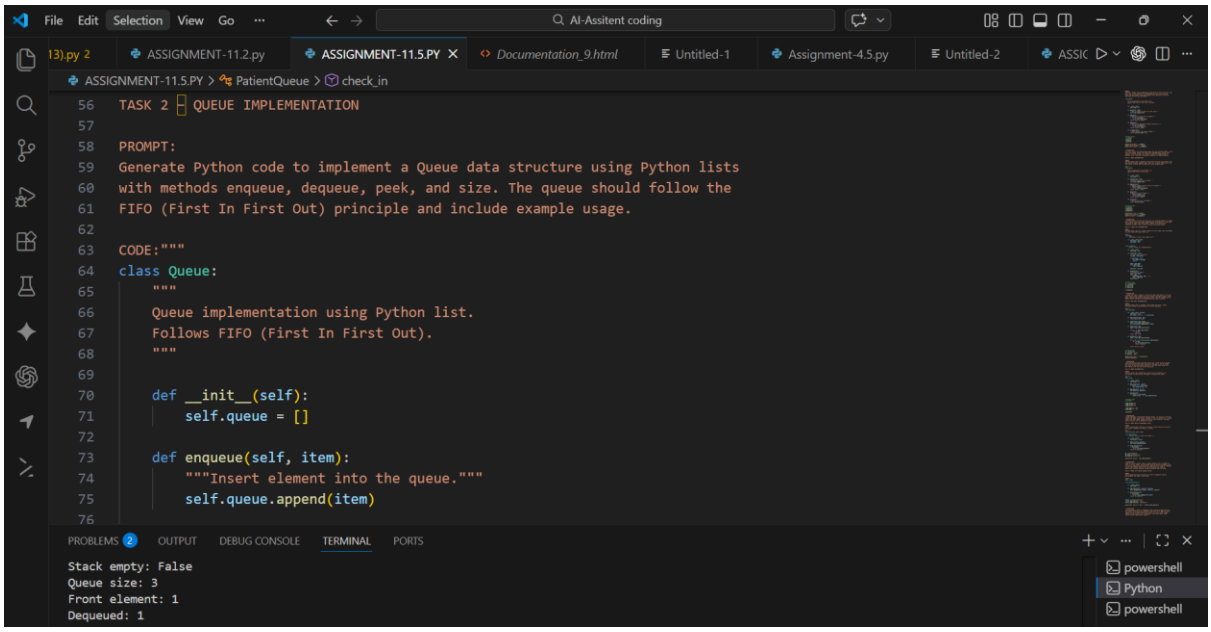
The stack follows the LIFO principle where the last inserted element is the first one removed. Python lists make stack implementation simple using append() for push and pop() for removal. Stacks are commonly used in function calls, recursion, expression evaluation, and undo operations.

TASK 2 – QUEUE IMPLEMENTATION

PROMPT:

Generate Python code to implement a Queue data structure using Python lists with methods enqueue, dequeue, peek, and size. The queue should follow the FIFO (First In First Out) principle and include example usage.

CODE:



OBSERVATION:

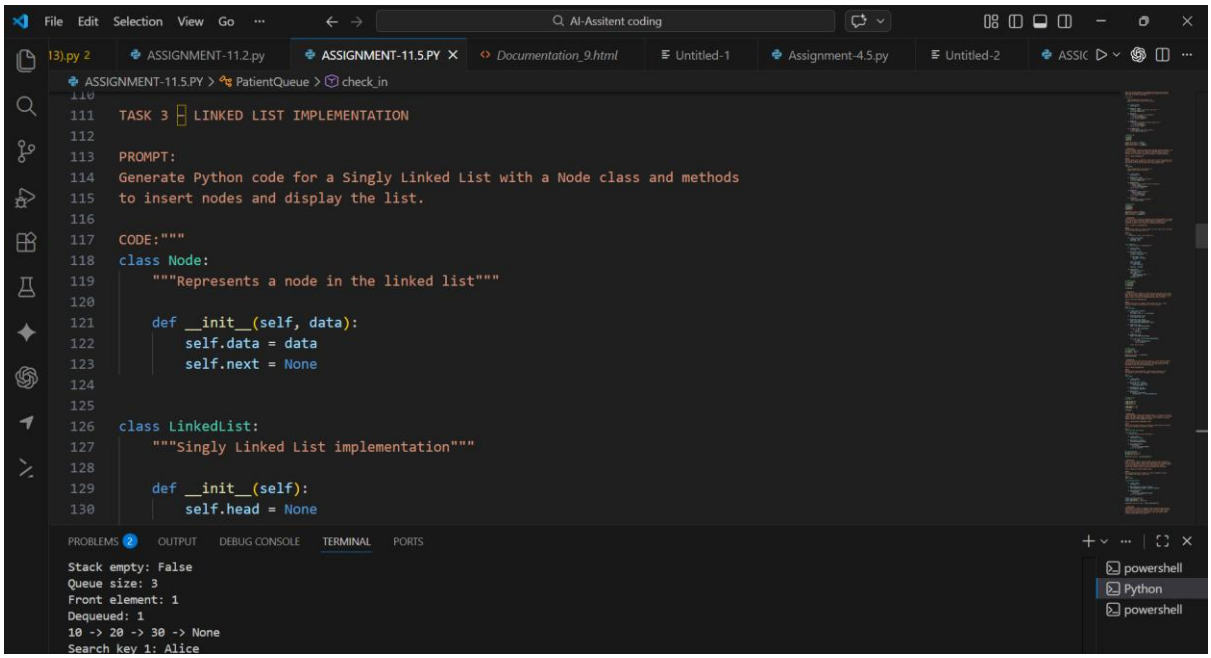
The queue follows the FIFO rule where the first inserted element is removed first. Python lists allow queue implementation using append() to add items and pop(0) to remove the front element. Queues are widely used in scheduling systems, printer queues, and task processing systems.

TASK 3 – LINKED LIST IMPLEMENTATION

PROMPT:

Generate Python code for a Singly Linked List with a Node class and methods to insert nodes and display the list.

CODE:



"OBSERVATION:

A linked list stores elements in nodes where each node points to the next node. Unlike arrays, linked lists do not require contiguous memory, which makes insertion and deletion operations easier. They are commonly used in dynamic memory allocation and implementing other data structures.

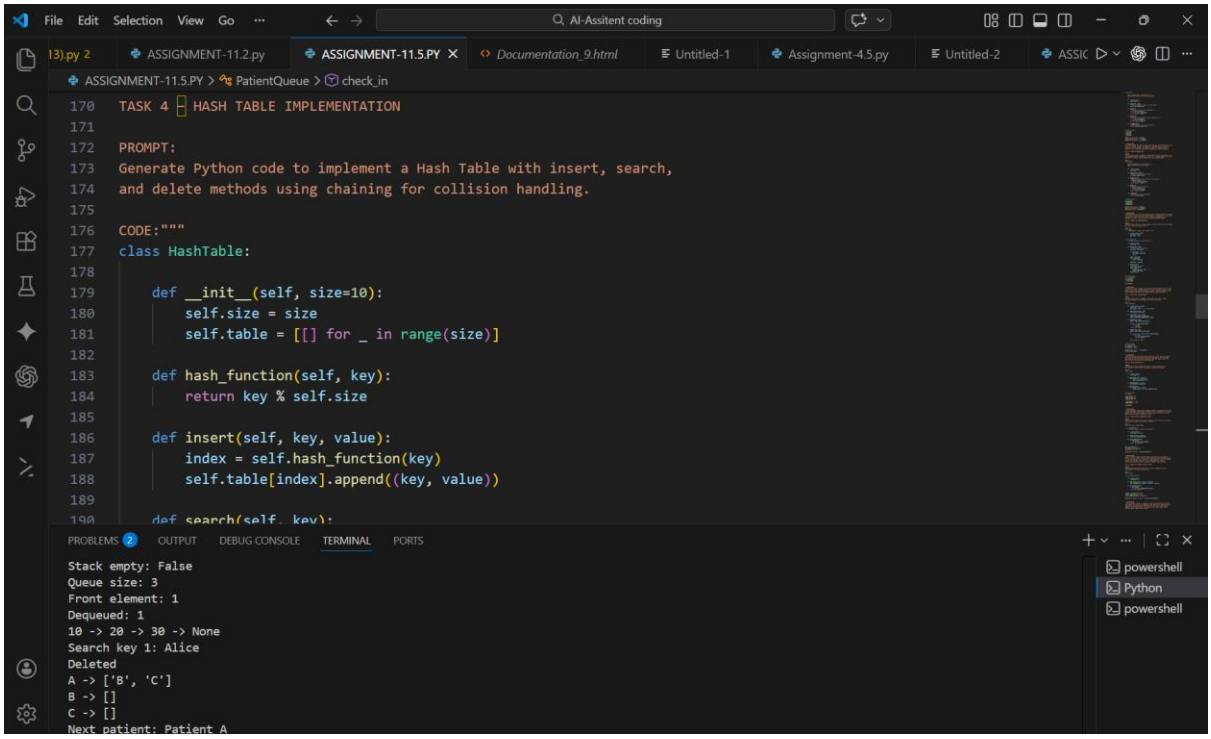
TASK 4 – HASH TABLE IMPLEMENTATION

PROMPT:

Generate Python code to implement a Hash Table with insert, search,

and delete methods using chaining for collision handling.

CODE:



OBSERVATION:

A hash table stores data using key-value pairs. A hash function converts

keys into indices where values are stored. Chaining is used to handle

collisions by storing multiple elements at the same index using lists.

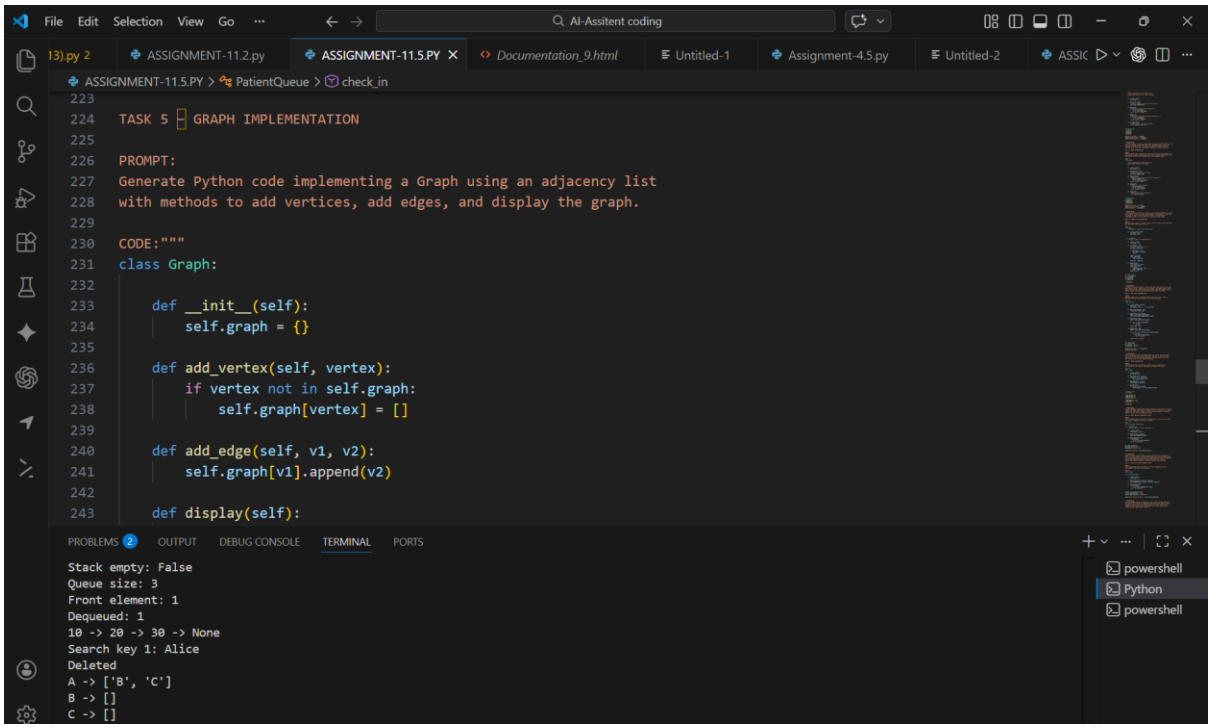
Hash tables allow very fast search operations.

TASK 5 – GRAPH IMPLEMENTATION

PROMPT:

Generate Python code implementing a Graph using an adjacency list with methods to add vertices, add edges, and display the graph.

CODE:



OBSERVATION:

A graph represents relationships between objects. It consists of vertices (nodes) and edges (connections). Adjacency lists are efficient for storing graphs with fewer edges. Graphs are used in social networks, road maps, network routing, and recommendation systems.

TASK 6 – SMART HOSPITAL MANAGEMENT SYSTEM

PROMPT:

Select suitable data structures for hospital system features and justify each choice. Implement one feature in Python.

CODE:

```
TASK 6 - SMART HOSPITAL MANAGEMENT SYSTEM

PROMPT:
Select suitable data structures for hospital system features and justify
each choice. Implement one feature in Python.

CODE:"""
from collections import deque

class PatientQueue:
    """Patient check-in system using queue."""

    def __init__(self):
        self.queue = deque()

    def check_in(self, patient):
        self.queue.append(patient)

    def treat_patient(self):
        if self.queue:
            return self.queue.popleft()
        return "No patients"


pq = PatientQueue()
pq.check_in("Patient A")
```

```
Next patient: Patient A
Next vehicle to pass: (1, 'Ambulance')
PS C:\Users\ARSHA THALLAPALLY\OneDrive\Desktop\AI-Assitent coding>
```

OBSERVATION:

Queue is used for patient check-in because patients must be treated in order of arrival (FIFO). Priority Queue would be used for emergency cases so critical patients are treated first. Hash tables are ideal for medical records since patient details can be retrieved quickly using ID.

Graphs help represent hospital navigation between rooms and wards.

TASK 7 – SMART CITY TRAFFIC CONTROL SYSTEM

PROMPT:

Select appropriate data structures for traffic management features and implement one feature using Python.

CODE:

```
TASK 7 - SMART CITY TRAFFIC CONTROL SYSTEM

PROMPT:
Select appropriate data structures for traffic management features
and implement one feature using Python.

CODE:"""
import heapq

class EmergencyTraffic:

    def __init__(self):
        self.queue = []

    def add_vehicle(self, priority, vehicle):
        heapq.heappush(self.queue, (priority, vehicle))

    def next_vehicle(self):
        if self.queue:
            return heapq.heappop(self.queue)
        return "No vehicles"


traffic = EmergencyTraffic()
traffic.add_vehicle(2, "Car")
traffic.add_vehicle(1, "Ambulance")
```

```
Next patient: Patient A
Next vehicle to pass: (1, 'Ambulance')
PS C:\Users\ARSHA THALLAPALLY\OneDrive\Desktop\AI-Assitent coding>
```

OBSERVATION:

Priority Queue is ideal for emergency traffic systems because vehicles with higher priority such as ambulances must move first. Regular traffic signals can use a Queue where vehicles pass in arrival order. Graphs represent road networks and intersections, while Hash Tables enable instant vehicle registration lookup